

COSC 423/523: Introduction to Artificial Intelligence

PA2: Multi-Agent Search in Pacman

Fall 2026

100 points

10% of course grade

Due: See Canvas

Overview

In this assignment, you will design agents for classic Pacman in a world that includes one or more ghosts. You will improve a reflex agent, implement minimax and expectimax search, add alpha-beta pruning, and design a stronger state-evaluation function. The Pacman engine, ghost agents, layouts, tree tests, and public autograder are provided.

Start from a fresh copy of the PA2 starter code rather than mixing it with files from an earlier assignment.

Starter folder: `Student_Starter_Code/multiagent`

File to edit and submit: `multiAgents.py`

Getting Started

Run commands from the starter folder shown above. To become familiar with the game and the supplied code, first play Pacman with the arrow keys and then run the provided reflex agent:

```
python pacman.py
```

```
python pacman.py -p ReflexAgent
```

```
python pacman.py -p ReflexAgent -l testClassic
```

Inspect `multiAgents.py` and make sure you understand the provided `ReflexAgent` before modifying it.

Project Files

File	Purpose
<code>multiAgents.py</code>	The only file you edit and submit; all of your multi-agent search agents and evaluation functions reside here.
<code>pacman.py</code>	Runs Pacman and defines the <code>GameState</code> interface used throughout the assignment.
<code>game.py</code>	Defines core types such as <code>Agent</code> , <code>AgentState</code> , <code>Directions</code> , and <code>Grid</code> .
<code>util.py</code>	Provides utility functions and data structures that may be useful.
<code>ghostAgents.py</code>	Provides the ghost controllers.
<code>layout.py</code> and layout files	Read and define Pacman boards.
<code>autograder.py</code> , <code>testParser.py</code> , <code>testClasses.py</code> , <code>multiagentTestClasses.py</code> , and <code>test_cases/</code>	Provide the public tests and autograding support.
Graphics, text-display, and keyboard files	Provide the graphical, text, and keyboard interfaces.

Do not change the support files. Do not rename provided functions or classes, and do not change required function signatures. The autograder depends on the supplied interface.

Using the Autograder

Run public tests without graphics using:

```
python autograder.py --no-graphics
```

Run one question, for example Question 2, using:

```
python autograder.py -q q2
```

```
python autograder.py -q q2 -no-graphics
```

Run one specific test using a command of the following form:

```
python autograder.py -t test_cases/q2/0-small-tree
```

The `-t` option displays graphics by default, whereas the `-q` option does not. Use `-graphics` to force graphics or `-no-graphics` to disable them.

Autograder note. The supplied autograder reports 25 legacy autograder points. Gradescope uses the 100-point allocation in this handout. The default command runs only the required questions. Public tests are a development aid; the correctness of the submitted implementation is the basis for the final grade, and course staff may review code individually when necessary.

General Instructions and Course Policy

- Work individually or in an instructor-approved team. A team submits one codebase using Gradescope's **group submission** feature and records its contribution statement as directed by the course staff.
- Use Python 3. Run commands from the project subfolder shown above. Do not rename required files or change function signatures.
- Do not share code across teams. Any material generative-AI use must be disclosed in the course AI Use Note, and every teammate must be able to explain the submitted implementation.
- The provided engine, layouts, utilities, and public tests are support code. Only the listed student file is graded unless this handout explicitly says otherwise.
- No separate report or presentation is required for the regular assignment grade.
- Submit your own work. Course staff may compare submissions for copied or logically redundant code. Follow the university and course academic-integrity policies.
- If you need help, use office hours, course meetings, and the course discussion forum. Do not post solution code, spoilers, or implementation details that would reveal answers to other students.

Required Work and Grading

Question 1: Reflex-Agent Warm-Up (10 PA points)

Improve `ReflexAgent.evaluationFunction` in `multiAgents.py` so that the supplied reflex agent plays respectably. This function evaluates a *state-action pair*: use the current state and the successor produced by the candidate action to judge that action.

- Use meaningful features, including food locations and ghost locations, and document the intent of the function.
- The supplied code demonstrates useful `GameState` queries. In particular, food grids provide an `asList()` method.
- Reciprocal features such as inverse distance can be more useful than raw distances when nearby objects should matter much more than distant ones. Handle zero distances safely.
- Printing the string representation of values such as `newGhostStates` can help you inspect the available data.
- Extensive tuning to a legacy maximum win-rate threshold is not required. Grading emphasizes meaningful features, clear intent, correct execution, and reasonable behavior.

Test on the small layout, then experiment with one or two ghosts on the default layout:

```
python autograder.py -q q1 -no-graphics
```

```
python pacman.py -p ReflexAgent -l testClassic
```

```
python pacman.py -frameTime 0 -p ReflexAgent -k 1
```

```
python pacman.py -frameTime 0 -p ReflexAgent -k 2
```

The default ghosts choose randomly. For additional experiments, use `-g DirectionalGhost` for directional ghosts, `-f` for a fixed random seed, `-n` to run several games, and `-q` to turn off graphics. A reflex agent may still lose frequently against two ghosts; do not spend disproportionate time tuning this warm-up.

Question 2: Minimax (25 PA points)

Implement `MinimaxAgent` in `multiAgents.py`. Pacman is agent 0 and acts as the maximizing agent. Ghosts act as minimizing agents and move in increasing agent-index order. Your implementation must work with an arbitrary number of ghosts, which means one max layer is followed by one min layer for each ghost.

- Implement the search recursively with one or more helper functions.
- Search to the depth in `self.depth`. One search ply is one Pacman move followed by every ghost's response. Increase the search depth only after the last ghost has moved.
- At a win, loss, depth limit, or other state with no legal action, return `self.evaluationFunction(state)`. Do not hard-code `scoreEvaluationFunction`; command-line options may select another evaluation function.
- Keep all nodes as full `GameState` objects obtained from `getAction` or `generateSuccessor`.
- Generate exactly the successors the algorithm needs. The public tree tests check the number of calls to `GameState.generateSuccessor`, so redundant calls and missing expansions will fail tests.

Use the small tree tests first, then run a Pacman game:

```
python autograder.py -q q2 -no-graphics
```

```
python pacman.py -p MinimaxAgent -l minimaxClassic -a depth=4
```

Useful reference values for the initial `minimaxClassic` state are 9, 8, 7, and -492 at depths 1, 2, 3, and 4, respectively. A correct minimax agent can lose some tests or move toward a ghost when it concludes that death is unavoidable; that behavior alone does not indicate an implementation error. You can inspect the latter behavior with:

```
python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3
```

On larger layouts, a shallow minimax agent may avoid death but fail to make progress. Question 5 addresses this limitation through a better evaluation function.

Question 3: Alpha–Beta Pruning (25 PA points)

Implement `AlphaBetaAgent` as a pruned version of the minimax search from Question 2. Extend alpha–beta pruning across all ghost layers. It must return the same minimax values as Question 2; action choices at tied values may depend on tie-breaking.

- Preserve the order returned by `GameState.getLegalActions`; do not reorder successors.
- Follow the project convention for *strict* pruning. At a max node, prune only when the value is strictly greater than beta. At a min node, prune only when the value is strictly less than alpha. **Do not prune on equality.**
- Do not call `GameState.generateSuccessor` more than necessary. The public tests compare expansion counts.

After checking the small tree tests, compare the Pacman behavior and speed with your minimax agent:

```
python autograder.py -q q3 -no-graphics
```

```
python pacman.py -p AlphaBetaAgent -a depth=3 -l smallClassic
```

The initial-state values on `minimaxClassic` should again be 9, 8, 7, and -492 at depths 1 through 4. Correct alpha–beta search may still lose some Pacman tests; the search result and expansion behavior, not whether Pacman wins every game, determine correctness.

Question 4: Expectimax (25 PA points)

Implement `ExpectimaxAgent`. Pacman remains a maximizing agent, but each ghost is modeled as choosing uniformly at random from its legal actions. Replace each ghost’s minimum with the arithmetic mean of the successor values.

- Support an arbitrary number of ghosts.
- Use the same agent ordering, terminal conditions, evaluation function, and depth convention as in minimax: the depth advances only after all ghosts have moved.
- Compute expectations with floating-point arithmetic and handle states with no legal actions as terminal states.

Debug on the small generic trees before testing in Pacman:

```
python autograder.py -q q4 -no-graphics
```

```
python pacman.py -p ExpectimaxAgent -l minimaxClassic -a depth=3
```

To observe the difference between an adversarial model and a random ghost model, compare these runs:

```
python pacman.py -p AlphaBetaAgent -l trappedClassic -a depth=3 -q -n 10
```

```
python pacman.py -p ExpectimaxAgent -l trappedClassic -a depth=3 -q -n 10
```

Expectimax may take calculated risks that minimax refuses because it models all legal ghost actions as equally likely rather than assuming the worst action. A correct expectimax agent can still lose individual games or tests.

Question 5: Better Evaluation Function (15 PA points)

Implement `betterEvaluationFunction` for game states. Unlike the reflex evaluation function in Question 1, this function receives a state rather than a state–action pair and is used to evaluate leaves of a look-ahead search tree.

- Combine the game score with sensible information about remaining food, distances to food, capsules, active ghosts, and scared ghosts.
- Treat wins, losses, collisions, and zero-distance reciprocal features carefully. Keep the function efficient because it is called at many search leaves.
- Grading emphasizes correct execution, a clear and documented design, and reasonable performance rather than a leaderboard rank or a legacy maximum-score threshold.

Test the evaluation function with depth-2 expectimax on `smallClassic` and one random ghost. The following commands run the public tests and a ten-game experiment:

```
python autograder.py -q q5 -no-graphics
```

```
python pacman.py -p ExpectimaxAgent -l smallClassic -a evalFn=better,depth=2 -q -n 10
```

Grading Summary

Question	PA Points
Question 1: Reflex-Agent Warm-Up	10
Question 2: Minimax	25
Question 3: Alpha–Beta Pruning	25
Question 4: Expectimax	25
Question 5: Better Evaluation Function	15
Total	100

What to Submit

- Submit `multiAgents.py` to the PA2 assignment on **Gradescope**.
- If you worked in an instructor-approved team, make one submission and use Gradescope’s **group submission**, and describe the division of work in the **Canvas assignment text entry** feature to add both teammates.
- Before submitting, run the required-scope autograder from a clean copy of the project folder and confirm that your code does not depend on modified support files.